
Time-Series Analysis of Global Temperature Data

Summer Internship Project Report

Student Name	Suchethan PH
Email	suchethan.ph.ug25@plaksha.edu.in
Course	Bachelor of Technology (B.Tech)
Batch	2025–2029
Institute Name	Plaksha University, Mohali, Punjab
Period of Internship	18th May 2026 – 14th June 2026
GitHub Repository	github.com/suchu-user/time-series-global-temperature
Report Submitted To	IDEAS — Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata

1. Abstract

This project performs an exploratory time-series analysis of historical global surface-temperature anomaly data, followed by a short unsupervised machine-learning exercise. Using the monthly global temperature dataset spanning 1880–2016 from two independent sources, GCAG and GISTEMP, the work covers data loading, datetime parsing, cleaning, feature extraction, rolling (moving-average) smoothing, grouping and aggregation, and pivot-table construction. The analysis confirms a clear long-term warming trend, with annual mean anomalies rising sharply in recent decades. The final section applies K-Means clustering to the scikit-learn digits dataset and evaluates the clusters against the true labels using a macro-averaged F1 score of approximately 0.79.

2. Introduction and Objectives

Global temperature *anomaly* datasets record how much warmer or cooler a period was relative to a long-term baseline average, rather than absolute temperatures. Studying these series is central to understanding climate change. The objectives of this notebook are to: (i) load and correctly structure the time-series data; (ii) engineer time-based features and smooth the series; (iii) summarise the data through grouping, aggregation and pivoting to surface seasonal and long-term patterns; and (iv) demonstrate a clustering workflow with an appropriate evaluation metric.

3. Dataset Description

The dataset `monthly_csv.csv` contains 3,288 monthly records with three columns: **Source** (GCAG or GISTEMP — two independent global temperature reconstructions), **Date** (monthly, 1880–2016), and **Mean** (the global mean temperature anomaly in degrees Celsius relative to a baseline). The data contains no missing values. As required by the internship guidelines, only this dataset is used for the analysis portion of the project.

4. Tools and Libraries

Python 3 with **pandas** and **numpy** for data manipulation, and **scikit-learn** (`load_digits`, `KMeans`, `f1_score`) for the clustering task. The work was carried out in a Jupyter Notebook.

5. Methodology and Implementation

Question 1 — Import Libraries and Load Data (2 marks)

The required libraries are imported and the CSV is read into `df_temp`. The first five rows confirm the three-column structure of the dataset.

CODE

```
import pandas as pd
import numpy as np

# Load the global temperature dataset (monthly_csv.csv) into a DataFrame
df_temp = pd.read_csv('monthly_csv.csv')
df_temp.head()
```

OUTPUT

	Source	Date	Mean
0	GCAG	2016-12-06	0.7895
1	GISTEMP	2016-12-06	0.8100

2	GCAG	2016-11-06	0.7504
3	GISTEMP	2016-11-06	0.9300
4	GCAG	2016-10-06	0.7292

Question 2 — Convert to Datetime and Sort

(1 mark)

The string `Date` column is parsed to a proper `datetime` type and the frame is sorted in ascending order, so the series now begins at the earliest record (January 1880).

CODE

```
# Convert the 'Date' column from string to datetime, then sort ascending
df_temp['Date'] = pd.to_datetime(df_temp['Date'])
df_temp = df_temp.sort_values('Date').reset_index(drop=True)
df_temp.head()
```

OUTPUT

	Source	Date	Mean
0	GISTEMP	1880-01-06	-0.3000
1	GCAG	1880-01-06	0.0009
2	GCAG	1880-02-06	-0.1229
3	GISTEMP	1880-02-06	-0.2100
4	GCAG	1880-03-06	-0.1357

Question 3 — Data Inspection

(2 marks)

The dataset has 3,288 rows and 3 columns and contains zero missing values, so no imputation is required.

CODE

```
print("Shape of the DataFrame:", df_temp.shape)
print("Total number of missing (null) values:", int(df_temp.isnull().sum().sum()))
```

OUTPUT

```
Shape of the DataFrame: (3288, 3)
Total number of missing (null) values: 0
```

Question 4 — Extract Year and Month

(1 mark)

Integer `Year` and `Month` columns are derived from the datetime column to enable time-based grouping later.

CODE

```
# Extract Year and Month from the datetime 'Date' column
df_temp['Year'] = df_temp['Date'].dt.year
df_temp['Month'] = df_temp['Date'].dt.month
df_temp.head()
```

OUTPUT

	Source	Date	Mean	Year	Month
0	GISTEMP	1880-01-06	-0.3000	1880	1
1	GCAG	1880-01-06	0.0009	1880	1
2	GCAG	1880-02-06	-0.1229	1880	2

3	GISTEMP	1880-02-06	-0.2100	1880	2
4	GCAG	1880-03-06	-0.1357	1880	3

Question 5 — 12-Month Moving Average

(2 marks)

A 12-month rolling mean of the anomaly is computed *separately for each Source* using `groupby` + `transform`, smoothing seasonal fluctuation. Early rows are NaN until a full 12-month window exists.

CODE

```
# 12-month moving average of 'Mean', computed separately for each Source
df_temp['Moving_Avg'] = (
    df_temp.groupby('Source')['Mean']
        .transform(lambda x: x.rolling(window=12).mean())
)
df_temp.tail()
```

OUTPUT

	Source	Date	Mean	Year	Month	Moving_Avg
3283	GCAG	2016-10-06	0.7292	2016	10	0.981917
3284	GISTEMP	2016-11-06	0.9300	2016	11	1.017500
3285	GCAG	2016-11-06	0.7504	2016	11	0.963992
3286	GISTEMP	2016-12-06	0.8100	2016	12	0.992500
3287	GCAG	2016-12-06	0.7895	2016	12	0.936292

Question 6 — Filter the Last 20 Years

(2 marks)

Records from the most recent 20 years (1997–2016 inclusive) are retained in `df_last20`, giving a shape of (480, 6).

CODE

```
# Keep only the last 20 years (inclusive of the most recent year)
latest_year = df_temp['Year'].max()
df_last20 = df_temp[df_temp['Year'] > latest_year - 20]
df_last20.shape
```

OUTPUT

(480, 6)

Question 7 — Group by Source

(2 marks)

Averaging the anomaly per source shows GCAG and GISTEMP agree closely on the long-run mean, validating the two independent reconstructions against each other.

CODE

```
# Average 'Mean' temperature for each data source
source_avg = df_temp.groupby('Source')['Mean'].mean()
source_avg
```

OUTPUT

Source	
GCAG	0.048797

```
GISTEMP    0.024380
Name: Mean, dtype: float64
```

Question 8 — Aggregate by Year

(1 mark)

Annual mean anomalies for the most recent years rise steadily, reaching their highest value in 2016 — clear evidence of recent warming.

CODE

```
# Average 'Mean' temperature per year
yearly_avg = df_temp.groupby('Year')['Mean'].mean()
yearly_avg.tail()
```

OUTPUT

```
Year
2012    0.629517
2013    0.661846
2014    0.742075
2015    0.882408
2016    0.964396
Name: Mean, dtype: float64
```

Question 9 — Hottest Month on Average

(2 marks)

Averaging the anomaly by calendar month identifies the month with the largest mean anomaly across the full record.

CODE

```
# Average 'Mean' temperature for each month across all years
monthly_avg = df_temp.groupby('Month')['Mean'].mean()
print(monthly_avg)
print("\nHottest month on average (month number):", monthly_avg.idxmax())
```

OUTPUT

```
Month
1      0.021871
2      0.029409
3      0.045972
4      0.034341
5      0.031106
6      0.022287
7      0.037955
8      0.039577
9      0.050192
10     0.052566
11     0.043543
12     0.030244
Name: Mean, dtype: float64

Hottest month on average (month number): 10
```

Question 10 — Pivot Table

(2 marks)

A Year $\times$ Month pivot of the last 20 years gives a compact grid of monthly anomalies, with any gaps filled by 0.

CODE

```
# Pivot: average 'Mean' per Month (columns) for each Year (rows)
pivot_table = df_last20.pivot_table(
    values='Mean', index='Year', columns='Month', aggfunc='mean', fill_value=0
)
pivot_table
```

OUTPUT

Month	1	2	3	4	...	9	10	11	12
Year					...				
1997	0.35230	0.39940	0.49085	0.41755	...	0.59250	0.64435	0.62325	0.61510
1998	0.60710	0.88300	0.63480	0.68790	...	0.48855	0.48325	0.45660	0.56980
1999	0.49035	0.66665	0.35510	0.39950	...	0.41380	0.40365	0.39440	0.51895
2000	0.30335	0.56195	0.57805	0.61055	...	0.42920	0.30135	0.29785	0.30960
2001	0.45600	0.42730	0.60465	0.54330	...	0.53250	0.51030	0.69400	0.53140
2002	0.72310	0.77230	0.84935	0.57625	...	0.60805	0.52365	0.59090	0.43330
2003	0.71035	0.55630	0.57010	0.55505	...	0.65060	0.73855	0.56420	0.74305
2004	0.59395	0.71660	0.66820	0.60700	...	0.53155	0.63885	0.74005	0.51405
2005	0.67115	0.54715	0.69090	0.71070	...	0.73335	0.74650	0.74400	0.65175
2006	0.51335	0.66105	0.61155	0.50885	...	0.63685	0.68370	0.68245	0.76960
2007	0.92195	0.68045	0.68340	0.74680	...	0.60875	0.58350	0.54360	0.49360
2008	0.25730	0.36870	0.74625	0.50635	...	0.59090	0.66950	0.66515	0.55175
2009	0.60775	0.54895	0.55470	0.63195	...	0.70415	0.64090	0.72080	0.63380
2010	0.71290	0.74400	0.88350	0.84935	...	0.59120	0.66840	0.77910	0.48970
2011	0.49030	0.51380	0.61575	0.65110	...	0.57990	0.64745	0.52285	0.54425
2012	0.44140	0.45675	0.55090	0.71175	...	0.74330	0.74110	0.72935	0.49775
2013	0.63365	0.59285	0.63695	0.54050	...	0.73285	0.68435	0.81965	0.68405
2014	0.71180	0.49500	0.76975	0.79030	...	0.84360	0.81755	0.67975	0.81040
2015	0.81205	0.87215	0.89845	0.75705	...	0.86800	1.02565	1.00275	1.11595
2016	1.11345	1.27105	1.26225	1.08165	...	0.87335	0.80960	0.84020	0.79975

[20 rows x 12 columns]

Question 11 — Load MNIST / Digits Data

(3 marks)

The scikit-learn digits dataset is loaded and split into features X (1797×64) and target y (1797).

CODE

```
from sklearn.datasets import load_digits

# Load the digits ("MNIST"-style) dataset and split into features/target
digits = load_digits()
X = digits.data
y = digits.target

print("Shape of features (X):", X.shape)
print("Shape of target (y):", y.shape)
```

OUTPUT

```
Shape of features (X): (1797, 64)
Shape of target (y): (1797,)
```

Question 12 — K-Means Clustering

(5 marks)

K-Means is fitted with 10 clusters (one per digit class) to produce a cluster label for every sample.

CODE

```
from sklearn.cluster import KMeans

# 10 clusters because there are 10 digit classes (0-9)
kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)

print("K-Means model fitted successfully.")
print("Cluster labels:", kmeans_labels)
```

OUTPUT

```
K-Means model fitted successfully.
Cluster labels: [1 3 3 ... 3 8 8]
```

Question 13 — F1 Score Evaluation

(5 marks)

Because cluster IDs are arbitrary, each cluster is mapped to its most frequent true digit before scoring. The macro-averaged F1 score is approximately 0.79.

CODE

```
from sklearn.metrics import f1_score

# K-Means cluster IDs are arbitrary, so map each cluster to the most
# frequent TRUE label inside that cluster before scoring.
mapped_labels = np.zeros_like(kmeans_labels)
for cluster in range(10):
    mask = (kmeans_labels == cluster)
    mapped_labels[mask] = np.bincount(y[mask]).argmax()

f1 = f1_score(y, mapped_labels, average='macro')
print("Macro-averaged F1 Score:", f1)
```

OUTPUT

```
Macro-averaged F1 Score: 0.7894620863890832
```

6. Results and Key Findings

- The two source reconstructions (GCAG and GISTEMP) produce very similar long-term average anomalies, increasing confidence in the data.
- Annual average anomalies climb markedly in recent years, peaking in 2016 — a clear long-term warming signal.
- The 12-month moving average smooths seasonal noise and makes the underlying upward trend easier to read.
- K-Means clustering of the digits dataset, after mapping clusters to true labels, achieves a macro-averaged F1 score of about 0.79, showing that unsupervised clustering recovers much of the digit structure but is imperfect.

7. Conclusion

This project demonstrated a complete, reproducible time-series workflow — from loading and cleaning through feature engineering, smoothing, aggregation and pivoting — applied to 137 years of global temperature anomaly data, and confirmed a pronounced recent warming trend. It then showed how an unsupervised K-Means model can be evaluated against ground-truth labels using a label-mapping step and a macro-averaged F1 metric. Together these cover the core exploratory-analysis and machine-learning skills targeted by the internship.